

InternLoom AI Hackathon

Manipal Institute of Technology
Problem Statement Document

Event Timing

8:30 AM Inauguration
4:30 PM Prize Distribution

Problem Statement

Smart Shortlisting Engine — Rank Resumes Against a Job Description

Context

Campus placement platforms like InternLoom don't just score one resume against one job — they take an entire pool of applicants and rank them against a single job opening before a human recruiter ever looks at them. At any real scale, this is a genuinely hard problem, not a simple search-and-filter task.

A strong candidate rarely describes themselves using the exact words in a job description. Someone who has “built REST APIs with Express and MongoDB” is clearly relevant to a Node.js backend role, even if the JD itself never uses the word “Express.” At the same time, specific tools and technologies named in a JD still matter — a role that explicitly asks for certain skills shouldn't be satisfied by only loosely related experience. On top of this, resumes vary widely in structure, phrasing, and completeness, and any ranking a system produces needs to be justifiable, not just a bare number.

Your task is to design and build a system that takes a Job Description and a batch of resumes, and returns a ranked, explainable shortlist of candidates — solving these challenges in whatever way your team thinks works best.

What You Must Build

Given one Job Description and a batch of 15–18 resumes (provided with this document), your system must:

1. Evaluate every resume against the JD using an approach that draws on both semantic search (matching based on meaning and context, not just literal words) and keyword search (matching explicit skills, tools, and terms). Both must genuinely factor into your final result. How you implement, weigh, or combine them is entirely your team's design decision — there is no fixed formula, technique, or library we are expecting to see.
2. Produce a ranked list of all candidates from best fit to worst fit, along with a final score for each.
3. For the top 3 ranked candidates, generate a short explanation of why they ranked there — which skills matched, and which required skills appear to be missing.

Important Constraint — Read Carefully

Simply pasting a resume and JD into an LLM API and asking it to “give this resume a score out of 100” does not meet the requirement and will not be scored under the core criteria. Your own system needs to genuinely perform the semantic and keyword matching as part of its logic — judges will ask each team to walk through how their matching actually works during evaluation.

Bonus Features (Optional — Extra Points Only)

- Flag potential bias or overly narrow phrasing in the JD itself that could unfairly exclude qualified candidates.
- Build a simple UI or chat layer where a “recruiter” can ask, e.g., “Why is Candidate X ranked above Candidate Y?” and get a natural-language answer.
- Handle messy or inconsistent resume formatting gracefully (typos, varied date formats, inconsistent section headers, etc.).

What You Are Provided

- 1 sample Job Description (Sample_JD.pdf) — Junior Full Stack Developer Intern role at a fictional company, TechNova Solutions.
- 18 sample resumes (PDF format) with deliberately varying degrees of fit to the JD — some strong matches, some partial/medium matches, and some weak matches — so your ranking output should show meaningful spread, not all-similar scores.

Judging Rubric

Criterion	Weight
Effective use of both semantic and keyword-based matching	35%
Quality and sensibility of the overall ranking produced	20%
Accuracy and clarity of top-3 candidate explanations	20%
Working end-to-end demo	15%
Bonus features	10%

Note: Exact evaluation/judging slot within the day will be communicated separately depending on the judging panel's availability.

Tips for Teams

- Start simple: get a basic end-to-end pipeline working first (even with a small subset of resumes), then improve matching quality and add bonus features.
- Solo participants should prioritize finishing the core requirement cleanly over attempting bonus features — a complete, working core solution will always score higher than an incomplete one with extras.
- Teams of 3–4 members can divide work: one member on matching/ranking logic, one on the explanation generator, one on data handling/parsing, one on demo/UI — but the working pipeline matters more than the interface.